



Python aplicado al Data Engineering

Como mencionamos en la clase anterior, ahora veremos Python como lenguaje principal para trabajar con los datos, ya que cuenta con un amplio ecosistema de herramientas orientadas al procesamiento, análisis e ingeniería de datos.

Antes de empezar a construir un pipeline de datos, necesitamos aprender algunas herramientas básicas de Python que nos permita **leer, manipular y almacenar información proveniente de archivos**.

Manejo de archivos con Python

Una de las tareas más comunes de un proceso de Data Engineering, consiste en obtener información desde diferentes fuentes:

- Archivos CSV (comma-separated-values): un archivo de texto plano separados por comas o puntos y coma.
- Archivos JSON (JavaScript Object Notation): Documento de texto plano que guarda información en pares claves y valor
- Bases de datos: SQL y NOSQL
- APIs: servicios Web que exponen datos.

Python proporciona la función **open()** para abrir y trabajar con archivos de forma local.

Python

```
archivo = open("archivo.json", "r")
```

El segundo parámetro de la función, indica el modo en el que queremos abrir el archivo. Algunos de los métodos más utilizados son:

- “r”: lectura.
- “w”: escritura.
- “a”: agregar contenido al final del archivo.

Sin embargo, cuando trabajamos con archivos en python, es recomendado utilizar la instrucción **with**.

Python

```
with open("archivo.json", "r") as archivo: #Con as asignamos un alias
    contenido = archivo.read()
```

El uso de **with** permite que Python se encargue de cerrar correctamente el archivo cuando termina la operación.

Si no utilizamos with, tenemos que cerrar manualmente la utilización del archivo, con:

Python

```
archivo.close()
```

Al utilizar la función close, si intentamos leer nuevamente el archivo, nos dará el siguiente error:

```
-----
ValueError                                Traceback (most recent call last)
Cell In[5], line 1
----> 1 archivo.read()

ValueError: I/O operation on closed file.
```

En ese punto, el archivo queda inhabilitado para realizar más operaciones, por lo que sería necesario reabrirlo mediante la función **open()**.

Para evitar este inconveniente, se recomienda utilizar el bloque **with**, ya que Python administra automáticamente la apertura y el cierre del archivo.

Trabajando con Archivos JSON con Python

En Ingeniería de datos, es frecuente trabajar con **datos semi estructurados** almacenados en formato JSON. Por ejemplo:

JSON

```
[
  {
    "cliente": "Juan Pérez",
    "ciudad": "Formosa",
    "peso": 15
  },
  {
    "cliente": "Ana Gómez",
    "ciudad": "Clorinda",
    "peso": 8
  }
]
```

En Python, podemos trabajar con estos tipos de datos utilizando un módulo llamado **json**

Python

```
import json

with open("archivo.json", "r") as archivo:
    contenido = json.load(archivo)
```

Una vez realizada la lectura del archivo, ya podemos trabajar con los datos utilizando las estructuras de python,

Python

```
print(len(contenido)) #con la función len() obtenemos la cantidad
de elementos de un arreglo.
print(type(contenido)) # con la función type, obtenemos el tipo
de dato de una variable.
print(contenido[0]) #accedemos al primer elemento de una lista
```

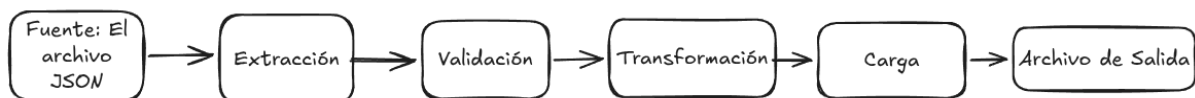


En este caso, al convertir el contenido del json, nos quedará una **lista de diccionarios**.

Creando un Pipeline de Datos con Python

En clases anteriores, construimos un pequeño proceso **ETL** utilizando JavaScript. El objetivo de la actividad era comprender de manera práctica como los datos pueden atravesar diferentes etapas desde que son obtenidos hasta que son almacenados nuevamente.

El flujo que habíamos construido puede representarse de la siguiente manera:



En este caso, vamos a seguir un ejemplo similar pero con Python.

Notebook:

<https://aprende.formosa.gob.ar/api/lessons/ea341d48-7fd3-49c4-a46a-c99f8772865a/media/8c3c5a95-52d4-4cc0-90f0-8a7a1a1fa943>

Datos:

<https://aprende.formosa.gob.ar/api/lessons/ea341d48-7fd3-49c4-a46a-c99f8772865a/media/dfc35b66-8588-4060-8fef-1ca6c64b5689>